
Comparing Flow Matching Representation Spaces for Image Generation

September 10, 2026

Andras Boguslaw Arancio

Abstract

Flow Matching (Lipman et al., 2023) is a generative modeling paradigm based on learning a vector field that induces a prescribed conditional probability path. Early image generation approaches operated directly in pixel space, typically parameterizing the vector field through UNet architectures (Ronneberger et al., 2015). Subsequent studies demonstrated the benefits of learning these fields in lower-dimensional latent representations (Dao et al., 2023). This study discusses and provides a comparative analysis of the two approaches.

1. Introduction

Flow Matching (Lipman et al., 2023) considers a source distribution p and a target distribution q , connected by a probability path p_t , $t \in [0, 1]$, such that $p_0 = p$ and $p_1 = q$. A time-dependent vector field u_t induces a flow φ_t through $\frac{d}{dt}\varphi_t(x) = u_t(\varphi_t(x))$, with $X_0 \sim p \Rightarrow \varphi_t(X_0) \sim p_t$.

Since the marginal vector field u_t is generally intractable, conditional probability paths $p_t(x|x_1)$ are introduced such that $p_t(x) = \int p_t(x|x_1)q(x_1)dx_1$. A suitable $p_t(x|x_1)$ yields a conditional field $u_t(x|x_1)$ available in closed form, making the learning problem tractable. The Conditional Flow Matching objective

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{t, x_1, x} [\|v_\theta(x, t) - u_t(x|x_1)\|^2] \quad (1)$$

shares gradients with respect to θ with the marginal objective, allowing a tractable training of the approximating field v_θ . New samples are generated by numerically integrating v_θ using an ODE solver.

We apply Flow Matching in two spaces: pixel space (Section 2) and a learned latent space (Section 3). Section 4 compares the two solutions. Experiments use the Smithso-

Email: Andras Boguslaw Arancio <arancio.1941005@studenti.uniroma1.it>.



Figure 1. Butterfly images generated using pixel-space Flow Matching, showing structural asymmetries.

nian Butterflies Subset (HugGAN Community, 2022), consisting of 1000 images resized to 128×128 pixels, split into 800 training and 200 test samples.

2. Pixel-space Flow Matching

In pixel-space Flow Matching, samples $x \sim q$ are represented directly as images. The velocity field v_θ is parameterized by a UNet (Ronneberger et al., 2015) suited for our purposes, extended with a time conditioning mechanism. Here, time is encoded through a sinusoidal embedding (Vaswani et al., 2017), scaled by a factor of 1000 to increase variation over $t \in [0, 1]$:

$$\begin{aligned} \text{PE}(t, 2i) &= \sin\left(\frac{1000t}{10000^{\frac{2i}{d_{\text{model}}}}}\right), \\ \text{PE}(t, 2i+1) &= \cos\left(\frac{1000t}{10000^{\frac{2i}{d_{\text{model}}}}}\right). \end{aligned} \quad (2)$$

The embedding is passed through an MLP, projected to the feature-map channels, and added to all spatial positions within each convolutional layer. As seen in Figure 1, generated butterflies generally exhibit visually pleasing wing patterns, though structural asymmetries are often present. This likely occurs because the vector field is learned directly in pixel space, where high-level semantic and structural features are not explicitly represented, making global properties like symmetry harder to capture.

3. Latent-space Flow Matching

In latent-space Flow Matching (Dao et al., 2023), images are first mapped to latent representations via an autoencoder. The vector field $v_\theta(x, t)$ is learned in latent space using a frozen encoder, while generated latents are mapped



Figure 2. Butterfly images generated using latent-space Flow Matching.

Table 1. Autoencoder reconstruction performance on test dataset.

Metric	Value
Test Reconstruction MSE	0.06872
Train Reconstruction MSE	0.00455

back to image space through the decoder. A ResNet-type autoencoder (He et al., 2016) was used. In the Appendix, training losses are to be found (Figure 7, over 600 epochs). We observed a train-test generalization gap of about one order of magnitude (Table 1). Since this behavior persisted with smaller architectures, the limited training set size is a more plausible explanation than overfitting. A pre-trained autoencoder was omitted to keep experiments self-contained. Although this poor generalization can propagate to the numerical flow approximation, the latent model remains competitive in Kernel Inception Distance. The latent vector field is parameterized by a substantially simpler MLP, significantly reducing computational cost. Using the same sinusoidal time embedding, generated samples (Figure 2) present noticeably fewer structural asymmetries. Architecture details are provided in the code.

4. Comparing the Models

The pixel-space model operates on images of dimension $3 \times 128 \times 128 = 49152$, whereas the latent model operates in dimension 1024. The following tests compare the two models.

Training Losses. Learning performances were compared using the relative loss:

$$\mathcal{L}_{\text{rel}} = \frac{\|v_\theta(x_t, t) - u_t\|_2^2}{\|u_t\|_2^2}. \quad (3)$$

Figure 3 compares the relative losses of both models (300 epochs, 16 batch size). Individual training dynamics are reported in Figures 4 to 6 in the Appendix.

Table 2. Training time comparison (300 epochs, 16 batch size).

Model	Training Time (min)
Pixel FM	80
Latent FM	9

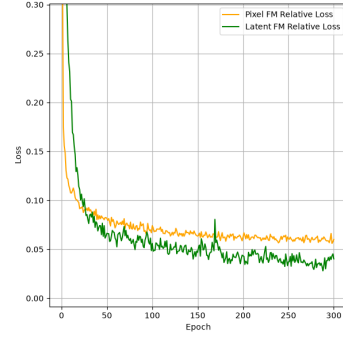


Figure 3. Latent-space vs pixel-space Flow Matching relative losses.

Table 3. Relative test loss comparison conducted over 10 iterations.

Model	Relative Loss
Pixel Flow Matching	0.05673 ± 0.00246
Latent Flow Matching	0.24060 ± 0.00468

Flow Approximation Relative Error. Table 3 reports relative test losses (3) comparing $v_\theta(x, t)$ with ground truth $u_t(x|x_1^{\text{Test}})$. The high autoencoder test reconstruction error impacts this metric, as unseen images map to representations distant from the training latent distribution.

Kernel Inception Distance. KID (Bińkowski et al., 2018) measures the distance between two distributions:

$$\text{KID}(P, Q) = \text{MMD}_k^2(\phi(P), \phi(Q)).$$

We computed the KID between test and generated distributions (in image space) using 100 subsets of size 75 (Table 4). Further details are provided in the Appendix.

Training Time. Table 2 shows time evaluation on 300 training epochs made on an Apple M2 Pro integrated GPU (19-core), the latent-space model trains significantly faster.

5. Conclusion

Overall, latent-space Flow Matching produces visually more pleasing and realistic results (Figure 8) than the pixel-space model (Figure 9), requiring considerably less training time. Numerical results are strongly influenced by the autoencoder. A better-generalizing autoencoder could further improve the latent model.

Table 4. Kernel Inception Distance comparison. Conducted on 100 subsets of size 75 of the test set.

Model	KID
Pixel FM	0.051110 ± 0.00557
Latent FM	0.051641 ± 0.00468

6. Statement on the use of AI

In this project, LLMs (ChatGPT 5 and Gemini 3) were used for general assistance in a careful and critical manner. The accuracy of all resulting outputs was always manually verified. In particular, AI was used for the following main purposes:

Code assistance. AI was mainly used to help the author make the best use of the PyTorch library, for bug discovery, correctness checking, and minor code cleanup under strict supervision. As a general workflow, the author designed the code architecture and workflow, wrote the code with partial assistance when using unfamiliar PyTorch modules, always referring to the PyTorch documentation as the primary source of information, and if needed revised the code in collaboration with LLMs.

Writing Matplotlib code. The author extensively used LLMs to write Matplotlib code for generating loss tracking plots from previously saved training data.

Writing test code. The author used LLMs to write tests, which primarily involved reusing previously written code for inference purposes.

Assistance in choosing the KID metric. The author sought assistance in determining the distribution distance metrics that best suited the present work. After carefully analyzing the advantages of the methodology proposed by the AI, the author proceeded to implement the suggested test metrics.

Translation and LaTeX assistance. The author used LLMs for translation purposes, bibliography curation, minor LaTeX code assistance, and standardizing the writing style of the present work (e.g., when to use uppercase letters).

References

- Bińkowski, M., Sutherland, D. J., Arbel, M., and Gretton, A. Demystifying mmd gans. In *International Conference on Learning Representations*, 2018.
- Dao, Q., Phung, H., Nguyen, B., and Tran, A. Flow matching in latent space. *arXiv preprint arXiv:2307.08698*, 2023.
- Detlefsen, N. S., Borovec, J., Schock, J., Harsh, A., Koker, T., Di Liello, L., Stancl, D., Quan, C., Grechkin, M., and Falcon, W. Torchmetrics - measuring reproducibility in pytorch. *Journal of Open Source Software*, 7(70):4101, 2022. doi: 10.21105/joss.04101.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE*

Conference on Computer Vision and Pattern Recognition, pp. 770–778, 2016.

HugGAN Community. Smithsonian butterflies subset. Hugging Face Datasets, 2022. URL https://huggingface.co/datasets/huggan/smithsonian_butterflies_subset. A 1,000-image processed subset of the Smithsonian Butterflies dataset.

Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. *International Conference on Learning Representations*, 2023.

Ronneberger, O., Fischer, P., and Brox, T. U-net: Convolutional networks for biomedical image segmentation. *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 234–241, 2015.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

Appendix

Kernel Inception Distance The KID is a metric used for measuring the distance between two distributions P and Q :

$$\text{KID}(P, Q) = \text{MMD}_k^2(\phi(P), \phi(Q)), \quad (4)$$

$$k(x, y) = \left(\frac{x^\top y}{d} + 1 \right)^3, \quad (5)$$

where the Maximum Mean Discrepancy (MMD) is computed as

$$\begin{aligned} \text{MMD}(P, Q) = & \mathbb{E}_{x, x' \sim P} [k(x, x')] + \mathbb{E}_{y, y' \sim Q} [k(y, y')] \\ & - 2\mathbb{E}_{x \sim P, y \sim Q} [k(x, y)]. \end{aligned} \quad (6)$$

The function ϕ represents a pretrained feature extraction model. KID was preferred over the more commonly used Fréchet Inception Distance (FID) due to the limited number of available samples. Unlike FID, whose estimation can be biased for finite sample sizes, KID provides an unbiased estimator based on Maximum Mean Discrepancy. This makes KID particularly suitable for evaluating the generated distribution in our relatively small-scale experimental setting. Details can be found at the TorchMetrics (Detlefsen et al., 2022) website : https://torchmetrics.readthedocs.io/en/v0.8.2/image/kernel_inception_distance.html.

More Experimental Results The following Tables and Figures contain training and test metrics conducted on the two models.

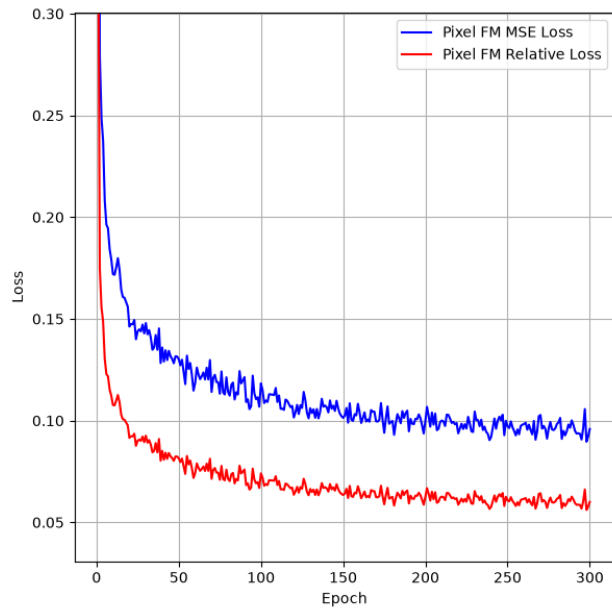


Figure 4. Pixel-space Flow Matching MSE loss and relative loss.

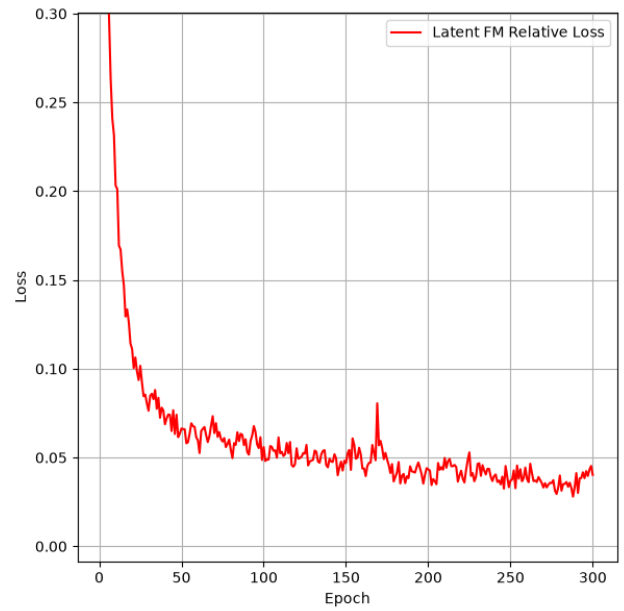


Figure 6. Latent-space Flow Matching relative loss.

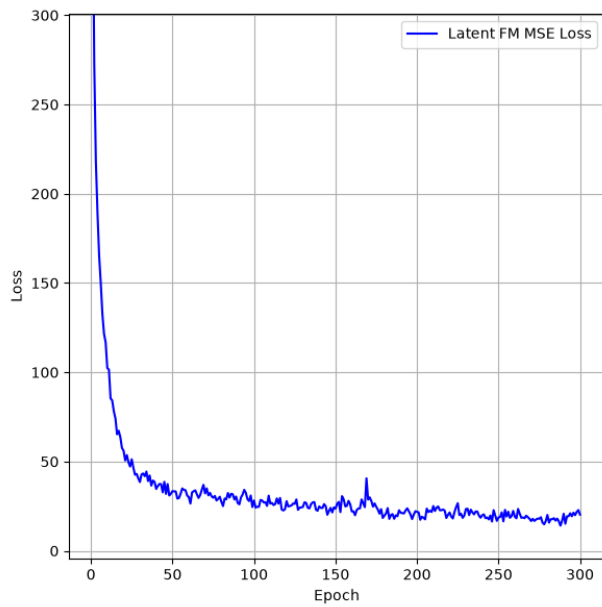


Figure 5. Latent-space Flow Matching MSE loss.

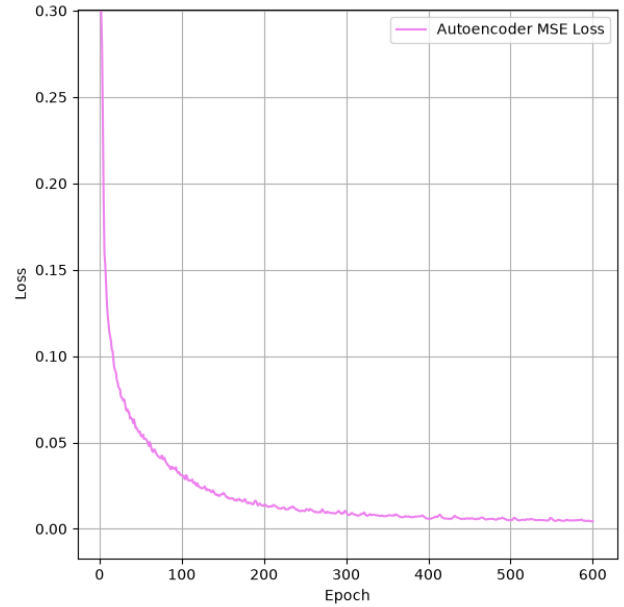


Figure 7. Autoencoder MSE reconstruction loss.

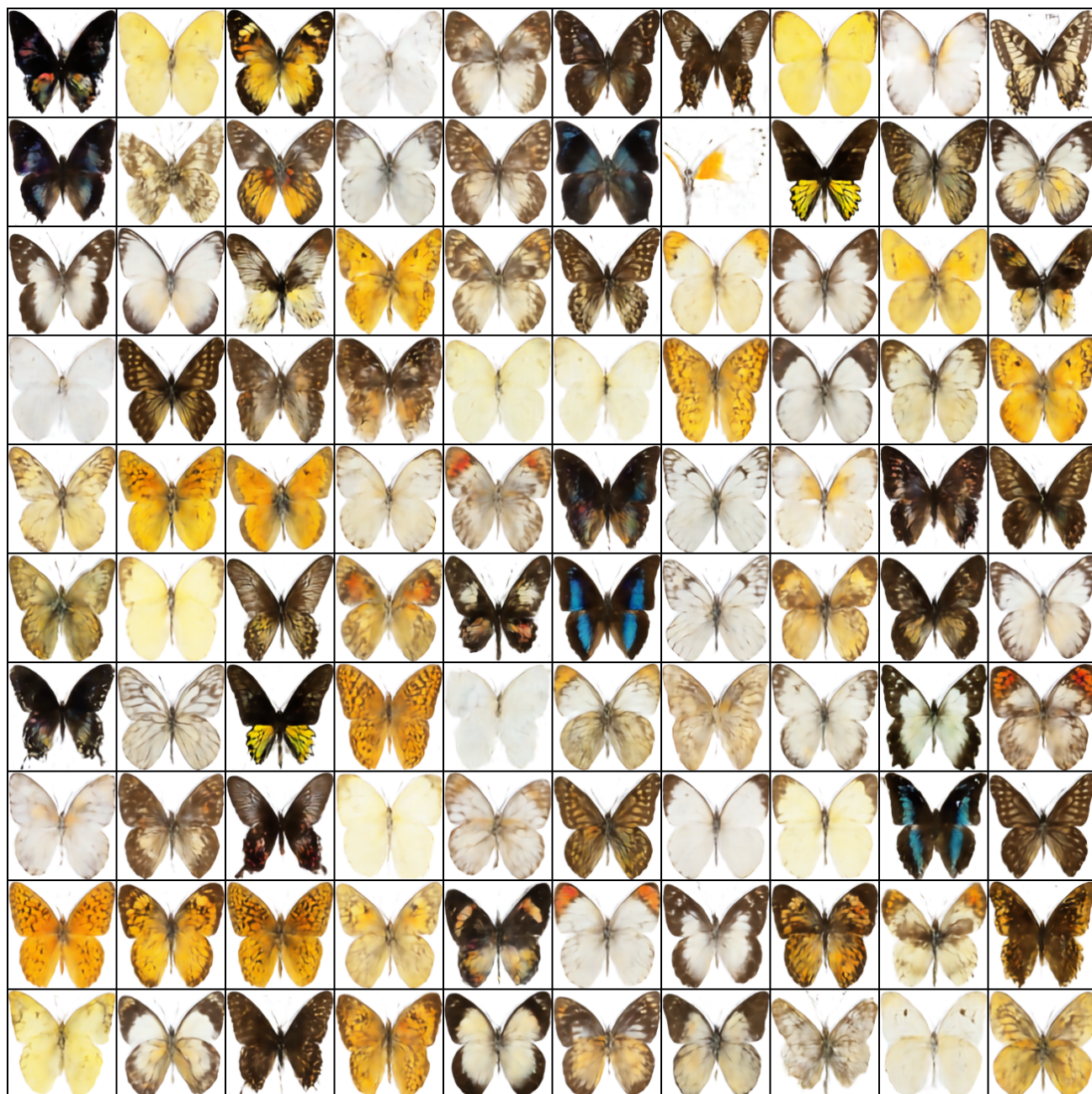


Figure 8. Butterfly images generated using latent-space Flow Matching.

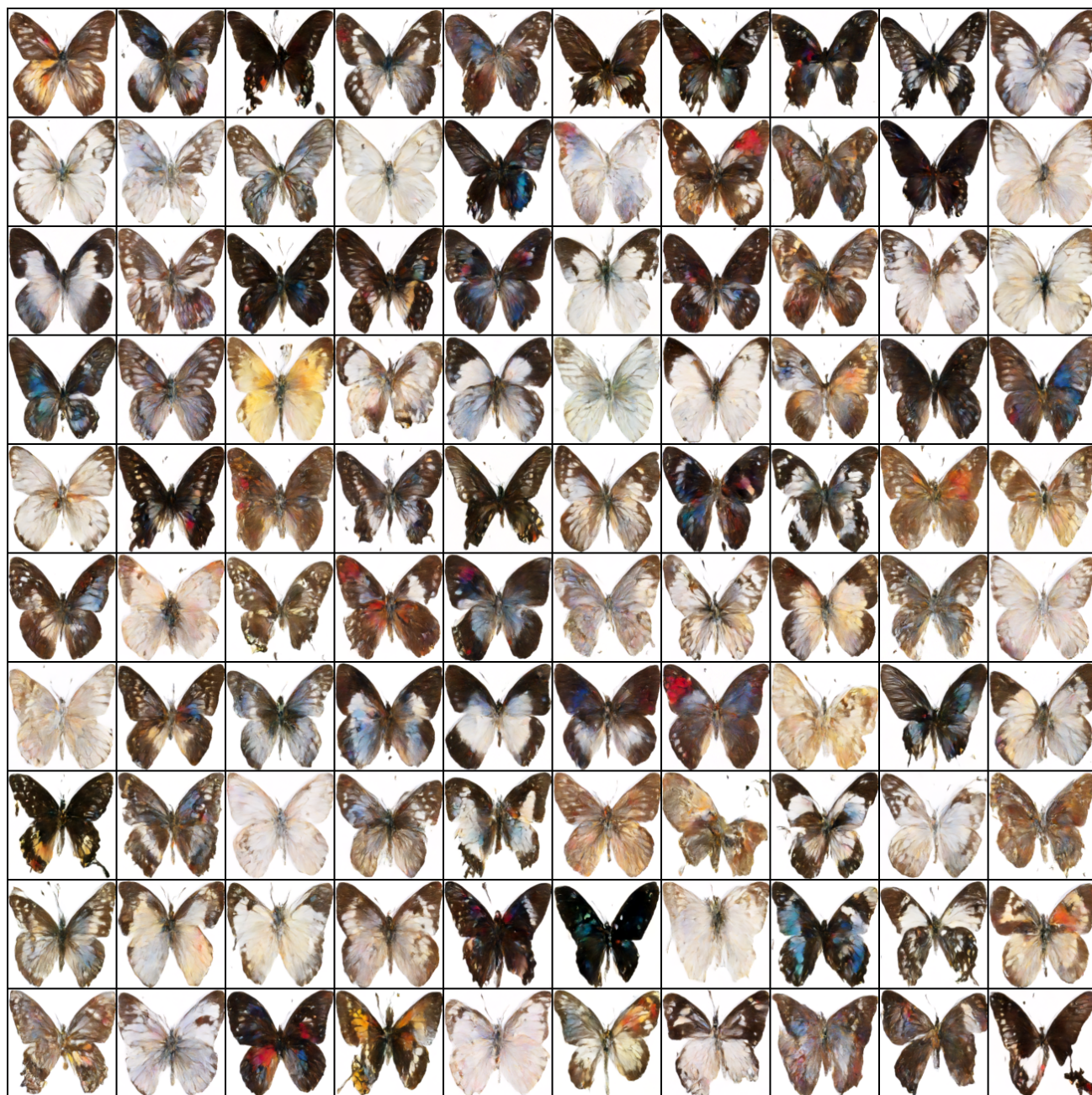


Figure 9. Butterfly images generated using pixel-space Flow Matching.